



TRAINING WORKSHEET · EXPLAINABLE MACHINE LEARNING

Understanding SHAP values

How SHAP explains what a model has learned — the theory, the practical mechanics, and the caveats worth stating in a write-up.

EDAM Capacity Building Programme · Supplementary reading

1. What SHAP is, and where it comes from

SHAP (**SHapley Additive exPlanations**) values are built on **Shapley values** from cooperative **game theory**, developed by Lloyd Shapley in **1953**. The original problem was how to divide a payout fairly among players in a coalition, based on each player's marginal contribution averaged over every possible order in which players could join.

Applied to machine learning, the “players” are the **features** and the “payout” is the difference between the model's prediction for a given instance and the average prediction across the dataset (the **baseline**). A feature's SHAP value is therefore its **average marginal contribution** to pushing that prediction away from the baseline, calculated across all possible combinations of the other features.

The practical consequence is that SHAP explains individual predictions, not just the model as a whole. Where a global importance score tells you which variables the model relies on overall, SHAP tells you why this particular patient, district or week received the prediction it did — which is usually what a policy audience actually wants to know.

2. Why SHAP is preferred over other approaches

SHAP is the **only additive feature attribution method** that satisfies all three of the following properties simultaneously:

- **Local accuracy** — the sum of all SHAP values plus the baseline exactly equals the model's actual prediction for that instance. Nothing is left unaccounted for.
- **Consistency** — if the model changes so that a feature's contribution increases, that feature's SHAP value never decreases.
- **Missingness** — a feature with no effect on the prediction is assigned a SHAP value of exactly zero.

This is a large part of why SHAP is favoured over LIME in applied work: LIME's local linear approximations do not guarantee this exact decomposition, so the contributions it reports need not sum back to the prediction being explained.

3. The computational problem, and how it is solved

Exact Shapley values require **evaluating every possible subset of features** — 2^n combinations — which is intractable beyond a handful of variables. Different SHAP “explainers” solve this in different ways, and choosing the right one for your model type is the difference between seconds and hours of computation.

Explainer	Model type	What it does
TreeExplainer	Tree-based (XGBoost, LightGBM, random forest)	Exact and fast (polynomial time), exploiting tree structure to avoid brute-force enumeration
KernelExplainer	Any model (model-agnostic)	Approximates Shapley values via weighted local linear regression, sampling feature coalitions — flexible but slow
DeepExplainer / GradientExplainer	Neural networks	Approximations tailored to deep models, using backpropagation-style computation
LinearExplainer	Linear and logistic models	Exact and fast, and can account for feature correlation if required

Table 1. Choosing the right SHAP explainer for your model.

4. Reading the standard plots

- **Force plot** — for a single prediction, shows each feature pushing the prediction up (red) or down (blue) from the baseline, in additive order.
- **Summary / beeswarm plot** — one point per instance per feature, showing both the SHAP value (impact) and the feature's actual value (colour). Useful for seeing not just that a feature matters, but the direction of its effect across the whole dataset.
- **Dependence plot** — the SHAP value for one feature plotted against that feature's actual value, often coloured by a second feature to reveal interactions.
- **Waterfall plot** — similar to a force plot but sequential and stacked; often the clearest option for explaining a single case in a written report.

5. Working in R

The **SHAPforxgboost** package is a common pairing with **xgboost** and produces summary, dependence and force plots with considerably **less manual wrangling** than working with raw SHAP values. The **fastshap** package is a more model-agnostic alternative.

If you are already extracting SHAP values from XGBoost fits, it is worth confirming you are using **TreeExplainer** under the hood — via `predcontrib = TRUE` in **xgboost**'s `predict` function, which is what most R wrappers call. That is the exact, fast path rather than a sampling-based approximation.

In Python, the **shap** package covers all explainer types directly, and **shap.Explainer** will select an appropriate one automatically based on the model object passed to it.

6. Caveats worth stating explicitly

- **Correlated features** — SHAP values can mislead when features are highly correlated, because the marginal contribution is split somewhat arbitrarily between them. Two near-duplicate variables might each receive roughly half of the “true” combined importance, making both look weaker than either really is.
- **Not causal** — a high SHAP value means the model relied heavily on that feature to make its prediction, not that the feature causally drives the outcome. This distinction is easy to blur in a write-up and is worth stating explicitly in any paper.
- **Baseline choice matters** — SHAP values are relative to the average prediction across your background dataset. Changing that reference set changes the numbers, so always state which background sample was used.
- **Explaining the model, not the world** — SHAP faithfully describes the model you fitted. If the model is poorly specified or trained on biased data, SHAP will explain those flaws just as confidently as it explains genuine signal.

“SHAP tells you what your model did. Whether what it did was sensible is still your judgement to make.”

References and further reading

- Lundberg, S. M. and Lee, S.-I. (2017) ‘A unified approach to interpreting model predictions’, *Advances in Neural Information Processing Systems*, 30, pp. 4765–4774.
- Lundberg, S. M., Erion, G., Chen, H. et al. (2020) ‘From local explanations to global understanding with explainable AI for trees’, *Nature Machine Intelligence*, 2(1), pp. 56–67.
- Shapley, L. S. (1953) ‘A value for n-person games’, in Kuhn, H. W. and Tucker, A. W. (eds.) *Contributions to the Theory of Games II*. Princeton: Princeton University Press, pp. 307–317.
- Molnar, C. (2022) *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 2nd edn. Available at: <https://christophm.github.io/interpretable-ml-book/>
- Ribeiro, M. T., Singh, S. and Guestrin, C. (2016) “‘Why should I trust you?’: explaining the predictions of any classifier”, *Proceedings of the 22nd ACM SIGKDD*, pp. 1135–1144.
- Liu, Y. and Just, A. (2023) **SHAPforxgboost**: SHAP plots for XGBoost. R package. Available at: <https://cran.r-project.org/package=SHAPforxgboost>
- Greenwell, B. (2024) **fastshap**: Fast approximate Shapley values. R package. Available at: <https://cran.r-project.org/package=fastshap>